



Factory & Abstract Factory Design Patterns

1. Why These Patterns Came Into Existence?

Before these patterns, **object creation logic** was often scattered throughout the codebase. That caused: - **Tight coupling** between classes and concrete implementations. - **Difficulty in extension**: Adding a new type meant modifying existing code everywhere. - **Violation of Open/Closed Principle (OCP)**: Code should be open for extension but closed for modification.

👉 The Factory patterns solve this by **decoupling object creation from usage**.

2. Factory Method Pattern

Definition

Defines an interface for creating an object, but lets subclasses decide which class to instantiate.

Example: Vehicle Factory

```
// Product
interface Vehicle {
    void drive();
}

// Concrete Products
class Car implements Vehicle {
    public void drive() {
        System.out.println("Driving a Car...");
    }
}

class Bike implements Vehicle {
    public void drive() {
        System.out.println("Riding a Bike...");
    }
}

// Creator (Factory)
abstract class VehicleFactory {
    abstract Vehicle createVehicle();
}

// Concrete Factories
class CarFactory extends VehicleFactory {
```

```

    Vehicle createVehicle() {
        return new Car();
    }
}

class BikeFactory extends VehicleFactory {
    Vehicle createVehicle() {
        return new Bike();
    }
}

// Client
public class Main {
    public static void main(String[] args) {
        VehicleFactory factory = new CarFactory();
        Vehicle v1 = factory.createVehicle();
        v1.drive();

        factory = new BikeFactory();
        Vehicle v2 = factory.createVehicle();
        v2.drive();
    }
}

```

👉 **Client doesn't depend on concrete classes**, only on abstraction.

When to Use Factory Method

- When a class cannot anticipate the type of objects it needs.
- When you want to delegate object creation to subclasses.
- To follow **OCP** (add new products without modifying client code).

When NOT to Use

- If object creation is simple and unlikely to change.
- If only one or two concrete types exist.

3. Abstract Factory Pattern

Definition

Provides an interface for creating **families of related objects** without specifying their concrete classes.

Example: GUI Toolkit (Windows vs Mac)

```
// Product Interfaces
interface Button {
    void render();
}

interface Checkbox {
    void render();
}

// Windows Products
class WindowsButton implements Button {
    public void render() {
        System.out.println("Rendering Windows Button");
    }
}
class WindowsCheckbox implements Checkbox {
    public void render() {
        System.out.println("Rendering Windows Checkbox");
    }
}

// Mac Products
class MacButton implements Button {
    public void render() {
        System.out.println("Rendering Mac Button");
    }
}
class MacCheckbox implements Checkbox {
    public void render() {
        System.out.println("Rendering Mac Checkbox");
    }
}

// Abstract Factory
interface GUIFactory {
    Button createButton();
    Checkbox createCheckbox();
}

// Concrete Factories
class WindowsFactory implements GUIFactory {
    public Button createButton() { return new WindowsButton(); }
    public Checkbox createCheckbox() { return new WindowsCheckbox(); }
}

class MacFactory implements GUIFactory {
    public Button createButton() { return new MacButton(); }
    public Checkbox createCheckbox() { return new MacCheckbox(); }
}
```

```
// Client
public class Application {
    private Button button;
    private Checkbox checkbox;

    public Application(GUIFactory factory) {
        button = factory.createButton();
        checkbox = factory.createCheckbox();
    }

    public void renderUI() {
        button.render();
        checkbox.render();
    }

    public static void main(String[] args) {
        GUIFactory factory;

        String os = "Windows"; // could be dynamic
        if (os.equals("Windows")) {
            factory = new WindowsFactory();
        } else {
            factory = new MacFactory();
        }

        Application app = new Application(factory);
        app.renderUI();
    }
}
```

👉 Client can switch **Windows/Mac** UI simply by changing the factory.

When to Use Abstract Factory

- When the system should be **independent of product creation**.
- When creating **families of related products**.
- When consistency across related products is required.

When NOT to Use

- If only one product type exists.
- If no product families are involved.
- If it adds unnecessary complexity.

4. Factory vs Abstract Factory

Aspect	Factory Method	Abstract Factory
Purpose	Delegates instantiation to subclasses	Provides interface to create families of related objects
Scale	Works on a single product	Works on multiple related products
Structure	Single method to create product	Multiple methods for related products
Example	CarFactory creates Car	WindowsFactory creates WindowsButton + WindowsCheckbox
Use Case	When varying one object creation	When requiring consistent families of objects

5. Real-World Applications

Factory Method

- Logging framework (FileLogger, ConsoleLogger, DBLogger)
- Document editor (WordDocument, PDFDocument, etc.)

Abstract Factory

- UI Toolkits (Windows/Mac/Linux styles)
- Database drivers (MySQLFactory, PostgreSQLFactory)
- Cross-platform libraries



Summary

- Use **Factory Method** → for creating one product at a time, letting subclasses decide which.
- Use **Abstract Factory** → for creating **families of related products**, ensuring consistency.